

02244 Logic for Security Project on Information Flow

- Hand-out: April 13, 2026
- Hand-in: via DTU Learn until May 11, 2026 **noon**
- We allow to work and hand-in in **groups of up to 3 students**.
- Each report must indicate which students are part of the group.
- The reports must be divided into **sections**, and each section must have **one** group member designated as **author**. This should reflect a fair distribution in report writing among the group members. Any section without such a marking of one single author will count as not submitted.
- The report must indicate which **resources** have been used to perform the work. This includes text books, research papers, information found on the web, the use of any AI tools, detailed suggestions from teachers, and results of discussions or cooperation with other students.
- Page Limit: 15 pages
- You may submit source codes as additional files, but the report must be entirely comprehensible without considering the source codes.
- **Presentation**: In the meeting on May 11th, we ask groups to present the protocols they have designed, so we can discuss the different solutions, common mistakes, and insights gained.

The Setting

In the first assignment, there was a server hosting data of a user (e.g. photos) and the user can give another service (e.g. a photobook service) access to their data in a secure way.

For this second assignment, we focus on the hosting server. The company who owns the hosting server wants to improve their service by offering three additional features beyond pure hosting:

- Users can give their data security labels (e.g. private, friends, public)
- Users can share data with other users according to the security labels (e.g. one can see the public-labeled data of all users, and the friends-labeled data of one's friends)
- Users can run programs on their data (e.g. a picture filter/enhancer on their photos). Thus, the server would directly execute a program (on behalf of the user) on the user's data. Note that these **programs cannot be trusted**: their programmers may have made honest mistakes or even deliberately included some malicious behavior.

Consider that this system should work for a variety of applications, like the following **hospital scenario**:

- A hospital wants to use the service to host medical data, where only particular people have read access or write access.
- The hospital, respectively the treating doctor, can run medical image processing programs on the medical data.
- One may also consider running a program for anonymizing patient records or making statistical evaluations, and after proper declassification giving researchers access to the results. However this point is not essential.

Of course, the company wants to offer a provably high level of security: even if users are dishonest or programs contain malicious code, the server never leaks information in violation of the security policy (that is given by the security labels). Similarly, if also integrity is considered, dishonest participants and malicious programs cannot alter trusted information (again according to the security policy).

Your Design

There is a wide variety of ways to set up such a server. Your mission—should you choose to accept it—is to provide a concept for the company how the new server with the additional features can be designed. The company is convinced that a good design can only work with information flow methods of the papers we treat in the course, i.e.:

- D. E. Denning, P. J. Denning: *Certification of Programs for Secure Information Flow*, Communications of the ACM 20(7), pp 504-513, ACM Press, 1977.
- D. Volpano, C. Smith, C. Irvine: *A sound type system for Secure Information Flow*, Journal of Computer Security 4(3), pp 167-197, IOS Press, 1996.
- A. C. Myers, B. Liskov: *A Decentralized Model for Information Flow Control*, ACM Symposium on Operating Systems Principles, ACM Press, 1997.

You can thus make use of the concepts in these articles, but only these concepts. In particular, if you want to use declassification, then you need to use the Myers and Liskov's Decentralized Model and their security labels. **Please do not invent your own custom variant of information flow analysis!** The only modification you are allowed to consider is to adapt the method to programming concepts not considered in the original papers (e.g., if you want to organize data in trees, you may define how trees are labeled).

You have a freedom to design the system in various ways:

- You could allow users to each define their own custom lattices, but you could also have one fixed lattice.
- You could allow for declassification of data, e.g., a hospital allows researchers access to patient data after anonymization.

- The software that can be run on the data to process it can be a fixed set of functions (e.g. image filters). Instead you could allow that users upload their own programs for image processing (e.g. medical image recognition).
- Programs could be handled as complete black boxes, i.e., the output may contain all the secrets of the input and if the software is untrusted, then its output is also untrusted.
- Alternatively, programs could be handled as white boxes, i.e., require that they are given in source code form, with all variables having security labels, so that an information flow analysis can be made before compiling and running the software.
- There can also be a combination between these two software models, where the white box programs can call black box programs as functions in any way that is guaranteed to uphold the security policy.
- You could choose to focus only on confidentiality, or to also allow for integrity.

Priorities

The most important thing for this assignment is **not** to support everything, but to have a precise concept that you can formally prove to be secure. So when in doubt, opt for a small set of simple and basic things your server supports and do those 100% right.

Server API

To make your design concrete, describe the interface that the server offers to each user. Here we assume that each user has already a secure connection to the server (e.g. TLS with some user authentication like in the first assignment).

The main point of this interface is that you **define a number of commands** that the user can send to the server such as uploading a file with and specify a security label for it. This interface, i.e., the set of commands and the responses from the server we call the Server API (Application Programming Interface).

Make a precise description of this interface:

- First: describe your security lattice if you have a fixed one; if you do not have a fixed one, then describe precisely what type of security lattice the users can choose and how that integrates with the security lattice of other users.
Note for Myers-Liskov you need to say what the set of actors is (who can be owner, reader, or writer).
- Describe the list of commands that the server accepts; you can write this like a function call where parameters have a name, a type, and if needed a security label.
- For each command, describe what the server should do, and what its response is (including possibly error messages). This can be descriptions in English, but

you can also supply this also with some (pseudo-) code explaining how the information is handled.

- Commands could involve uploading programs, and the server performing an information flow analysis. In this case, specify precisely what the server should check: what is the security lattice, which of the methods from class would be used, and how the program can be invoked by the user (e.g. as a function with certain arguments).
- Give for each API command a precise argument that executing this command cannot lead to an illegal information flow.
In particular, if you allow for uploading programs as source code and the server executes the program only after a successful information flow analysis, then you can use in your argument the guarantees that the information flow analysis ensures. So please be precise what exactly is checked, what guarantees you get from this, and how this integrates with the rest of the server.

This is the main task of this report—the analysis that your design is secure no matter what dishonest users and malicious code try to do!

The Report

The work must be documented in a report, again maximal 15 page, that documents the following:

- A description of the participants/roles in the system.
- A description of the security goals that you want to ensure, both confidentiality and integrity.
- Your design choices and reasoning: what security goals you want to achieve, and how the information flow achieves them.
- The server API, i.e., list of commands and description how they work.
- A precise argument for each API command that they ensure the information flow.
- The justification for all declassifications (if you perform any).

This is an individualized group report, i.e., the report has to be partitioned into sections/subsections such that **each part has a unique author** and the report must that is clearly denote the authorship.

Each report must indicate which resources have been used to perform the work. This includes text books, research papers, information found on the web, detailed suggestions from teachers, and results of discussions or discussions with other students not part of the group.